

Lab 11: Implement Job Sequencing with Deadlines

Theory:

The Job Sequencing Problem is a classic optimization challenge categorized under Greedy Algorithms. Given a set of n jobs where each job i has a deadline $d_i \geq 1$ and a profit $p_i \geq 0$, the objective is to find a sequence that maximizes total profit.

The Greedy Strategy:

To maximize profit, we must prioritize jobs with the highest earnings. However, simply picking the highest profit isn't enough; we must also manage time slots. A job is ideally scheduled at the latest possible time slot before its deadline. This "late-scheduling" strategy is crucial because it leaves earlier slots open for jobs with even tighter deadlines.

Complexity Overview

Resource	Complexity	Condition
Time	$O(n^2)$	Scanning n slots for n jobs
Space	$O(n)$	Tracking array for slots/result

Algorithm

1. Sort all given jobs in descending order of their profit.
2. Find the maximum deadline among all jobs to determine the size of the time slot array.
3. Initialize the time slot array to empty (FALSE).
4. Iterate through the sorted jobs. For each job, try to place it in the latest available time slot that occurs on or before its deadline.
5. If a valid slot is found, mark it as filled and add the job's profit to the total profit.
6. Repeat until all jobs are checked or all slots are filled.

Pseudocode

```
PROCEDURE JobSequencing(Jobs[], n)
    // Step 1: Optimization Setup
    Sort(Jobs) based on Profit in DESCENDING order

    max_d ← FindMaxDeadline(Jobs)
    slot ← Array of size [max_d] initialized to FALSE
    result ← Array of size [max_d]
    total_profit ← 0

    // Step 2: Greedy Slot Allocation
    FOR i ← 0 TO n - 1 DO
        // Try to schedule job at its latest possible time
        FOR j ← MIN(max_d, Jobs[i].deadline) - 1 DOWNT0 0 DO
```

```
    IF slot[j] is FALSE THEN
        slot[j] ← TRUE
        result[j] ← Jobs[i].id
        total_profit ← total_profit + Jobs[i].profit
        BREAK // Job scheduled, move to next job
    END IF
END FOR
END FOR

RETURN result, total_profit
END PROCEDURE
```

Conclusion:

The experiment successfully implements and verifies the Job Sequencing algorithm using a greedy paradigm. The jobs were sorted dynamically by their associative profit in descending order to prioritize high-value execution.

The algorithm evaluated the 5 input jobs across a maximum deadline threshold of 3. Job A (profit: 100) was securely scheduled on Slot 2, followed by Job C (profit: 27) on Slot 3, and finally Job D (profit: 25) seized the remaining Slot 1. The cumulative scheduling layout flawlessly yielded the maximum possible optimal total profit of 152, verifying the structural soundness of the late-scheduling greedy method.